

Laboratory 7

TASK: MULTITHREADED APPLICATION USING CONDITION VARIABLES (C++)

This laboratory develops understanding of **Condition Variables** in C++. Students will use condition variables to coordinate threads, send signals, and safely manage shared ticket sales in a multithreaded ticket-selling simulation **on Linux/MacOS**.

Scenario is the same as in Laboratory 6

Tasks:

- 1. Create a multithreaded C++ ticket-selling program.**
 - Use `std::thread` to create multiple seller threads. (1 point)
- 2. Use a Condition Variable to coordinate thread behavior.**
 - Use `std::condition_variable` with `wait()` and `notify_one()` / `notify_all()`.
 - Threads should wait for signals before attempting to sell tickets. (1.5 points)
- 3. Use a Mutex along with the Condition Variable.**
 - Required for the protection of shared data. (0.5 point)
- 4. Prevent overselling and ensure safe access to shared variables.**
 - Carefully structure the condition-variable loop with checks such as `while (tickets > 0) condition.wait(lock)` (0.5 point)
- 5. Pass multiple arguments via a struct.**
 - Must include: Thread ID, Movie title, current time and date
 - Threads must print these information upon running. (0.5 point)
- 6. Simulate delay using `sleep_for()`.**
 - Introduce realistic timing in the selling process. (0.5 point)
- 7. Measure performance and compare execution time with Lab 6.** (0.25 point)
- 8. Comment your code thoroughly.** (0.25 point)

NOTE: Upload the print-screen of final code execution on enauczanie and of course your code in C++ :)